Proteomics-Based Patient Stratification of Pediatric Brain Tumours

CPTAC/CHOP Brain Cancer Cohort (brain_cptac_2020)

2026-04-18

Contents

Overview . . . 1
1. Packages & Configuration . . . 2
2. Data Loading . . . 3
3. Quality Control . . . 5
4. Dimensionality Reduction (PCA to NMF input) . . . 7
5. Selecting the Number of Clusters (k) . . . 8
6. Consensus NMF Clustering . . . 9
7. Survival Analysis . . . 12
8. Biological Characterisation . . . 18
9. Optional: NBS Comparison Arm . . . 23
References . . . 29

Overview

This notebook reproduces and extends the proteomics-based patient stratification from Petralia et al. (Cell, 2020) using the publicly available CPTAC/CHOP pediatric brain cancer dataset (`brain_cptac_2020` on cBioPortal).

Dataset: 218 tumour samples from 199 patients across 7 histological subtypes (LGG, Ependymoma, HGG, Medulloblastoma, Ganglioglioma, Craniopharyngioma, ATRT), each with WGS, RNA-seq, global proteomics, and phosphoproteomics.

Why proteomics?
Unlike sparse binary mutation profiles (which need network smoothing to be informative), proteomics data is already a dense, continuous, pathway-integrated signal. It directly captures the functional downstream consequences of mutations, splicing, and post-translational regulation. In a sense, the "smoothed" output that NBS tries to recover from mutations.

Pipeline summary:

```
Local data files
      load + clean column names
    v
Quality control
      remove high-missing proteins, KNN imputation, PCA sanity check
    v
Dimensionality reduction
      top 25 PCs, shift to non-negative for NMF
    v
k selection
      cophenetic correlation, dispersion, silhouette across k = 2-8
    v
```

```
Consensus NMF clustering
      60 runs per k, consensus matrix, final cluster labels
    v
Biological validation
       Kaplan-Meier survival (log-rank test)
       Histological composition per cluster
       Differential protein expression (limma one-vs-rest)
```

---

1. Packages & Configuration

```r
# Core
library(tidyverse) # dplyr, ggplot2, tidyr, purrr, stringr
library(janitor) # clean_names() for consistent column naming
library(car) # model performance metrics
library(gridExtra) # arrange output

# Matrix / imputation
library(impute) # KNN imputation for missing proteomics values (Bioconductor)

# Clustering
library(NMF) # Non-negative matrix factorisation + consensus clustering
library(cluster) # silhouette scores
library(pheatmap) # annotated heatmaps
library(RColorBrewer) # colour palettes

# PCA visualisation (only used for k selection)
#library(factoextra) # fviz_eig() scree plot helper

# Survival
library(survival) # Surv(), survfit(), coxph()
library(survminer) # ggsurvplot()

# Differential expression
library(limma) # linear models for proteomics; well-validated on MS data
```

```r
# Global parameters
set.seed(42)

DATA_DIR <- "./brain_cptac_2020" # folder with downloaded cBioPortal files
MISSING_THRESHOLD <- 0.30 # remove proteins missing in > 30% of samples
N_PCS <- 25 # PCA components fed into NMF (same as paper)
K_MIN <- 2 # minimum k to test
K_MAX <- 8 # maximum k to test
NMF_RUNS <- 60 # NMF initializations per k (more = more stable, 60 matches paper)
NMF_RUNS_9 <- 5 # for faster computation in section 9

cat("Configuration ready. Data directory:", DATA_DIR)
```

```
## Configuration ready. Data directory: ./brain_cptac_2020
```

---

2. Data Loading

We read four flat files downloaded from cBioPortal. All files live in `DATA_DIR`. Column names are cleaned with `janitor::clean_names()` for consistent `snake_case`.

```r
# Clinical (patient-level)
clin_raw <- read.delim(
  file.path(DATA_DIR, "data_clinical_patient.txt"),
  comment.char = "#", sep = "\t", header = TRUE, check.names = FALSE,
  stringsAsFactors = FALSE
) |> janitor::clean_names()

# Clinical (sample-level)
samples_raw <- read.delim(
  file.path(DATA_DIR, "data_clinical_sample.txt"),
  comment.char = "#", sep = "\t", header = TRUE, check.names = FALSE,
  stringsAsFactors = FALSE
) |> janitor::clean_names()

# Proteomics (proteins × samples z-scores)
prot_raw <- read.delim(
  file.path(DATA_DIR, "data_protein_quantification.txt"),
  comment.char = "#", sep = "\t", header = TRUE, check.names = FALSE,
  stringsAsFactors = FALSE
) |> janitor::clean_names()

# Somatic mutations
mut_raw <- read.delim(
  file.path(DATA_DIR, "data_mutations.txt"),
  comment.char = "#", sep = "\t", header = TRUE, check.names = FALSE,
  stringsAsFactors = FALSE
) |> janitor::clean_names()

cat("Files loaded.\n")
```

```
## Files loaded.
```

```r
cat("  Clinical patients :", nrow(clin_raw), "rows\n")
```

```
##   Clinical patients : 199 rows
```

```r
cat("  Clinical samples:", nrow(samples_raw), "rows\n")
```

```
##   Clinical samples: 218 rows
```

```r
cat("  Proteomics rows:", nrow(prot_raw), "(should equal n_proteins)\n")
```

```
##   Proteomics rows: 6429 (should equal n_proteins)
```

```r
cat("  Mutations rows:", nrow(mut_raw))
```

```
##   Mutations rows: 9951
```

```r
# Build proteomics matrix (samples × proteins)
# The file is formatted as proteins × samples with the first column being the
# protein identifier. We transpose so rows = samples, columns = proteins.
# R prepends 'x' to numeric-starting column names; we strip that and convert
# underscores back to hyphens to match the sample IDs in the sample file.
```

```r
prot_wide <- prot_raw |>
  column_to_rownames("composite_element_ref") |> # protein IDs to rownames
  select(where(~ sum(!is.na(.)) > 0)) |> # drop all-NA columns
  as.matrix() |> t() # now: samples × proteins

storage.mode(prot_wide) <- "numeric"


# Fix rownames: "x7316_1781" to "7316-1781"
rownames(prot_wide) <- rownames(prot_wide) |>
  str_remove("^x") |>
  str_replace_all("_", "-")


# Parse survival from patient-level clinical
# os_status is formatted as "1:DECEASED" or "0:LIVING"
clinical <- clin_raw |>
  mutate(
    os_event = as.integer(str_detect(toupper(os_status), "DECEASED|1:DECEASED")),
    os_months = suppressWarnings(as.numeric(os_months)))


# Build unified sample table
# Joins sample-level info (histology, tumour type) with patient-level survival
samples <- samples_raw |>
  select(sample_id, patient_id, cancer_type_detailed,
         tumor_type, age_at_specimen_diagnosis) |>
  left_join(
    select(clinical, patient_id, os_months, os_event, sex),
    by = "patient_id")

cat("Proteomics matrix:", nrow(prot_wide), "samples ×", ncol(prot_wide), "proteins\n")
```

```
## Proteomics matrix: 218 samples × 6429 proteins
```

```r
cat("Patients with OS data:", sum(!is.na(clinical$os_months)), "\n")
```

```
## Patients with OS data: 192
```

```r
cat("Events (deaths):", sum(clinical$os_event, na.rm = TRUE), "\n")
```

```
## Events (deaths): 40
```

```r
cat("Histological subtypes:")
```

```
## Histological subtypes:
```

```r
samples |> count(cancer_type_detailed, sort = TRUE) |> print()
```

```
##                        cancer_type_detailed  n
## 1               Pediatric Low Grade Gliomas 93
## 2                         Ependymomal Tumor 32
## 3              Pediatric High Grade Gliomas 25
## 4                           Medulloblastoma 22
## 5                             Ganglioglioma 18
## 6 Craniopharyngioma, Adamantinomatous Type 16
## 7          Atypical Teratoid/Rhabdoid Tumor 12
```

---

3. Quality Control

Two systematic issues in mass-spectrometry proteomics must be addressed before clustering:

1. **Missing values**: proteins below the detection limit are absent. Missingness is non-random (low-abundance proteins drop out more). We remove proteins missing in > `MISSING_THRESHOLD` of samples, then impute remaining NAs with KNN imputation.
2. **Outlier samples**: samples missing > 40% of the remaining proteins likely reflect technical failures.

```r
# Per-protein and per-sample missing rates
missing_per_protein <- colMeans(is.na(prot_wide))
missing_per_sample <- rowMeans(is.na(prot_wide))

# Distribution of missing rates across proteins
data.frame(missing_rate = missing_per_protein) |>
  ggplot(aes(x = missing_rate)) +
  geom_histogram(bins = 60, fill = "#4E79A7", colour = "white") +
  geom_vline(xintercept = MISSING_THRESHOLD,
             colour = "firebrick", linetype = "dashed", linewidth = 0.8) +
  annotate("text",
           x = MISSING_THRESHOLD + 0.02, y = Inf,
           label = paste0("Remove if > ", MISSING_THRESHOLD * 100, "% missing"),
           colour = "firebrick", vjust = 2, hjust = 0, size = 3.5) +
  labs(title = "Missing value rate per protein",
       subtitle = "Proteins to the right of the dashed line are excluded",
       x = "Fraction of samples with missing value", y = "Number of proteins") +
  theme_bw(base_size = 12)
```

```r
# 1. Remove high-missing proteins
prot_filtered <- prot_wide[, missing_per_protein <= MISSING_THRESHOLD]
cat(sprintf("Proteins removed (> %d%% missing): %d | Remaining: %d\n",
            as.integer(MISSING_THRESHOLD * 100),
            ncol(prot_wide) - ncol(prot_filtered),
            ncol(prot_filtered)))
```

```
## Proteins removed (> 30% missing): 0 | Remaining: 6429
```

```r
# 2. Remove outlier samples
outlier_samples <- names(missing_per_sample[missing_per_sample > 0.40])
if (length(outlier_samples) > 0) {
  cat("Outlier samples removed:", paste(outlier_samples, collapse = ", "), "\n")
  prot_filtered <- prot_filtered[!rownames(prot_filtered) %in% outlier_samples, ]
} else {
  cat("No outlier samples detected.\n")}
```

```
## No outlier samples detected.
```

```r
# 3. KNN imputation for remaining NAs
# impute.knn() expects features as rows, samples as columns → transpose twice.
# k = 10 neighbours is standard for proteomics datasets of this size.
cat("Running KNN imputation (k = 10)...\n")
```

```
## Running KNN imputation (k = 10)...
```

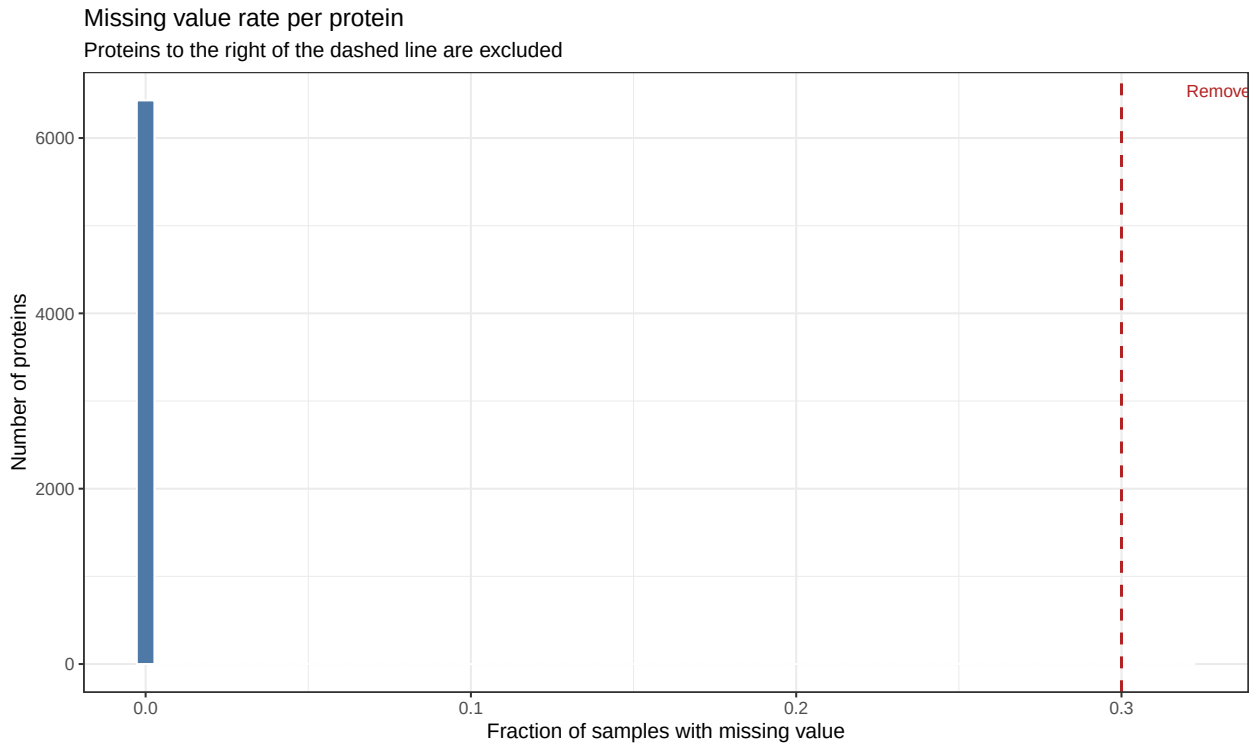


Figure 1: Proteins to the right of the dashed line are removed.

```
prot_imputed <- prot_filtered |>
  t() |>
  impute.knn(k = 10) |>
  _$data |> t()
```

```
## Cluster size 6429 broken into 5848 581
## Cluster size 5848 broken into 3800 2048
## Cluster size 3800 broken into 678 3122
## Done cluster 678
## Cluster size 3122 broken into 22 3100
## Done cluster 22
## Cluster size 3100 broken into 1999 1101
## Cluster size 1999 broken into 934 1065
## Done cluster 934
## Done cluster 1065
## Done cluster 1999
## Done cluster 1101
## Done cluster 3100
## Done cluster 3122
## Done cluster 3800
## Cluster size 2048 broken into 1275 773
## Done cluster 1275
## Done cluster 773
## Done cluster 2048
## Done cluster 5848
## Done cluster 581
```

```r
cat("Final matrix after QC:", nrow(prot_imputed), "samples ×",
    ncol(prot_imputed), "proteins")
```

```
## Final matrix after QC: 218 samples × 6429 proteins
```

---

4. Dimensionality Reduction (PCA to NMF input)

Running NMF directly on ~6,000 protein columns is noisy and slow. We first:

1. **Scale** each protein to unit variance (z-score across samples)
2. **Apply PCA** and retain the top `N_PCS` = 10 components
3. **Shift** PC scores to be non-negative (add |min|), as required by NMF

This linear shift preserves all pairwise distances.

```r
# Scale proteins before PCA (proteomics data should be z-scored column-wise)
#prot_scaled <- scale(prot_imputed)
prot_scaled <- prot_imputed # manual temporary fix until we decide what to do

# PCA
pca_res <- prcomp(prot_scaled, scale. = FALSE)
var_expl <- round(100 * pca_res$sdev^2 / sum(pca_res$sdev^2), 2)

cat(sprintf("Top 3 PCs explain %.1f%% of total variance.\n",
            sum(var_expl[1:3])))
```

```
## Top 3 PCs explain 45.8% of total variance.
```

```r
cat(sprintf("Top %d PCs explain %.1f%% of total variance.",
            N_PCS, sum(var_expl[1:N_PCS])))
```

```
## Top 25 PCs explain 76.8% of total variance.
```

```r
data.frame(PC = seq_along(var_expl), variance = var_expl) |>
  filter(PC <= 100) |>
  ggplot(aes(x = PC, y = variance)) +
  geom_line(colour = "#4E79A7") +
  geom_point(size = 1.5, colour = "#4E79A7") +
  geom_vline(xintercept = N_PCS, linetype = "dashed",
             colour = "firebrick", linewidth = 0.8) +
  annotate("text", x = N_PCS + 1, y = max(var_expl) * 0.75,
           label = paste0("Retain top ", N_PCS, " PCs"),
           colour = "firebrick", hjust = 0, size = 3.5) +
  labs(title = "Scree plot: variance explained per PC",
       x = "Principal Component", y = "Variance explained (%)") +
  theme_bw(base_size = 12)
```

```r
# Visualise PC1 vs PC2, coloured by histological subtype
pca_df <- as.data.frame(pca_res$x[, 1:5]) |>
  rownames_to_column("sample_id") |>
  left_join(select(samples, sample_id, cancer_type_detailed), by = "sample_id")

ggplot(pca_df, aes(x = PC1, y = PC2, colour = cancer_type_detailed)) +
  geom_point(alpha = 0.75, size = 2.5) +
  labs(title = "PCA of QC-passed proteomics",
       subtitle = "Colour = histological subtype",
```

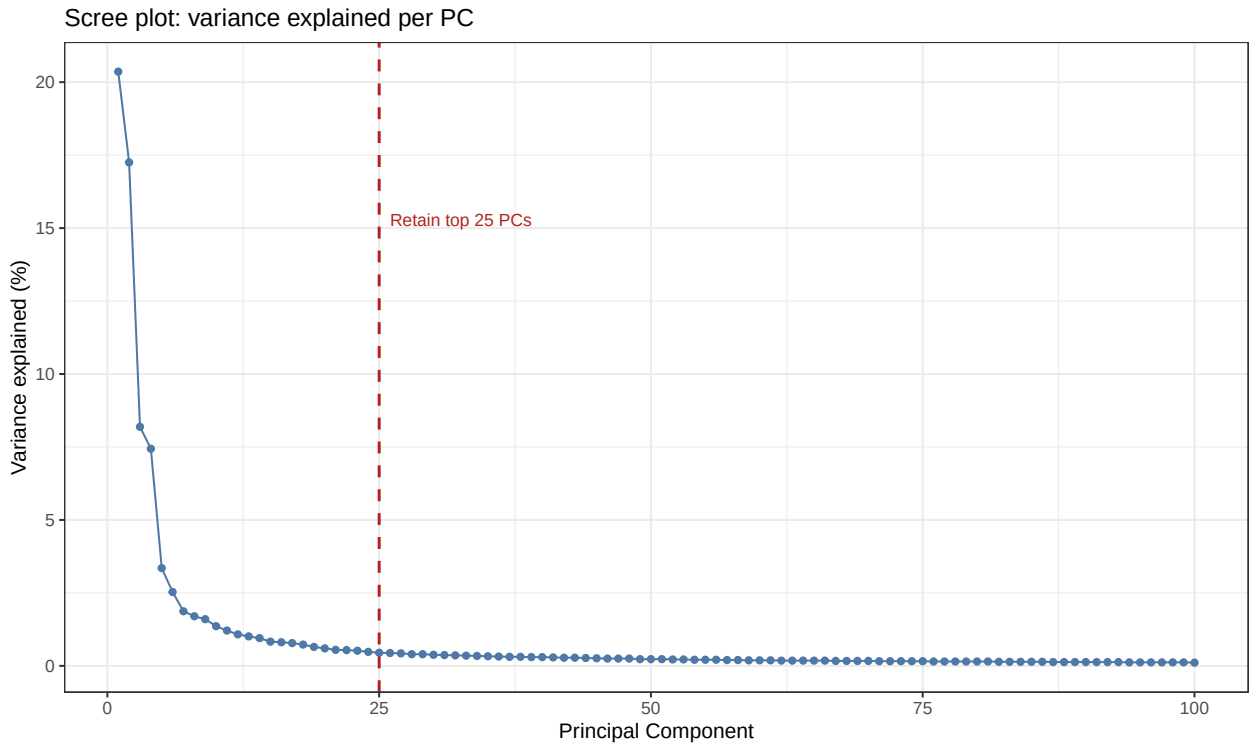


Figure 2: Scree plot: most signal is captured in the first 10 PCs.

```
       x = paste0("PC1 (", var_expl[1], "% var)"),
       y = paste0("PC2 (", var_expl[2], "% var)"),
       colour = "Subtype") +
  theme_bw(base_size = 12)
```

```
# Extract top N_PCS PC scores and shift to non-negative for NMF
pca_scores <- pca_res$x[, 1:N_PCS]
pca_scores_nn <- pca_scores - min(pca_scores) # shift smallest value to 0

cat("NMF input matrix:", nrow(pca_scores_nn), "samples ×",
    ncol(pca_scores_nn), "PCs\n")
```

```
## NMF input matrix: 218 samples × 25 PCs
```

```
cat("Value range: [", round(min(pca_scores_nn), 3), ",",
    round(max(pca_scores_nn), 3), "]")
```

```
## Value range: [ 0 , 82.315 ]
```

---

5. Selecting the Number of Clusters (k)

NOTE

Since the current automatic detection tends to select low values for k (which don't work well with the rest of the pipeline), the notebook has been manually set to use the same k value as the paper (`k = 8`). This number of clusters k was determined by evaluating the cophenetic correlation coefficient across possible k values. `k = 8` showed the highest coefficient, indicating the most robust and stable clustering where samples

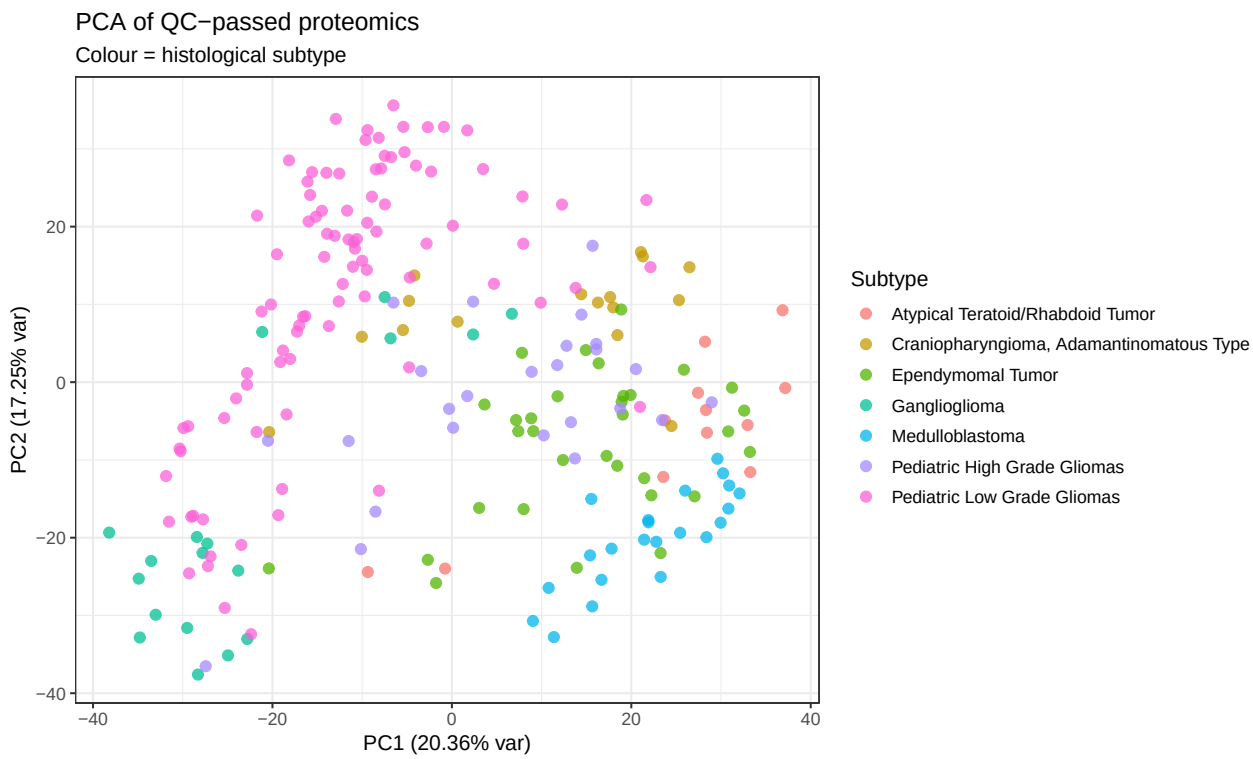


Figure 3: PCA coloured by histological subtype. Distinct subtypes (e.g. ATRT, MB) should form loose clusters; overlap reflects genuine molecular similarity across diagnoses, a core finding of the original paper.

reliably grouped together. This should aid in our analysis. Some chunks are now useless, as their purpose was to aid in k selection, so they've been turned off. To run and see them in the Knit, remove `eval=FALSE` and `echo=FALSE`.

Automated k selection

We sweep k from `K_MIN` to `K_MAX` and evaluate three complementary stability metrics from the NMF consensus:

- **Cophenetic correlation**: how faithfully the consensus matrix preserves sample distances (close to 1 = stable)
- **Dispersion**: how bimodal the consensus matrix is (values near 0 or 1)
- **Silhouette score**: how well-separated clusters are in feature space

The best k shows a consistent peak or elbow across all three. We prefer the silhouette score as the primary criterion because it directly measures biological cluster separability.

```
# Manual override:
# To use the k from the published paper (k = 8) instead, uncomment:
BEST_K <- 8
```

---

6. Consensus NMF Clustering

With `BEST_K` chosen, we run a final NMF and extract the consensus matrix and cluster labels.

Interpreting the consensus heatmap:

- Dark blue diagonal blocks indicate patients always cluster together (stable)
- Light off-diagonal regions indicate patients from different clusters never co-cluster (good separation)
- Mixed histologies within a cluster indicate cross-boundary molecular grouping, a key finding of the original paper

```
nmf_best <- nmf(
  t(pca_scores_nn), # transpose: features (PCs) as rows, samples as columns
  rank = BEST_K,
  nrun = NMF_RUNS,
  seed = 42,
  method = "brunet",
  .options = "vr")
```

```
## Runs: |                                                          Runs: |
## System time:
##    user  system elapsed
##  50.547   1.939  27.208
```

```
# Extract consensus matrix
cons_matrix <- consensus(nmf_best) # 218×218 expected
# Assign each sample to the cluster with its highest NMF basis weight.
# We use basis() rather than predict() here: predict() assigns to the
# component with the highest H-matrix value, which can collapse to a single
# cluster when one component dominates. basis() scores are per-sample and
# more reliably reflect the consensus structure.
basis_mat <- basis(nmf_best) # features × k (25 × BEST_K)
coef_mat  <- coef(nmf_best) # k × samples (BEST_K × 218)

# Cluster assignment: argmax across k for each sample (column of coef_mat)
cluster_labels <- apply(t(coef_mat), 1, which.max)
names(cluster_labels) <- colnames(coef_mat)  # real sample IDs are now column names

cat("cons_matrix dim:", dim(cons_matrix), "\n")  # should be 218 218
```

```
## cons_matrix dim: 218 218
```

```
cat("Cluster distribution:\n")
```

```
## Cluster distribution:
```

```
print(table(cluster_labels))
```

```
## cluster_labels
##  1  2  3  4  5  6  7  8
## 43 11  3 39  1 48 44 29
```

```
# Fix: rename cons_matrix rows/cols to match real sample IDs
# pca_scores_nn rownames and cons_matrix rows are in the same order,
# so we can directly assign them
#rownames(cons_matrix) <- rownames(pca_scores_nn)
#colnames(cons_matrix) <- rownames(pca_scores_nn)

# NOTE: `basis_mat` used to be called `basis`, in case of downstream issues.
```

```
# Annotation data frame for the heatmap
sample_ann <- tibble(sample_id = names(cluster_labels),
```

```r
                      cluster = paste0("C", cluster_labels)) |>
  left_join(select(samples, sample_id, cancer_type_detailed), by = "sample_id") |>
  column_to_rownames("sample_id")

# Colour palettes
cluster_colours <- setNames(
  brewer.pal(max(BEST_K, 3), "Set2")[1:BEST_K],
  paste0("C", 1:BEST_K))
subtypes_present <- unique(sample_ann$cancer_type_detailed)
subtype_colours <- setNames(
  colorRampPalette(brewer.pal(8, "Set3"))(length(subtypes_present)),
  subtypes_present)

# Sort by cluster using the cons_matrix row order
# to guarantee the annotation and matrix rows stay aligned
sample_order <- order(cluster_labels[rownames(cons_matrix)]) # match cons_matrix
cons_matrix_ordered <- cons_matrix[sample_order, sample_order]

# Build ordered annotation FIRST, keyed to the reordered matrix rows
ordered_sample_ids <- rownames(cons_matrix)[sample_order]
ann_ordered <- sample_ann[ordered_sample_ids, , drop = FALSE]

# Plot with ordered matrix
pheatmap(
  cons_matrix_ordered, # use ordered matrix
  cluster_rows = FALSE, cluster_cols = FALSE,
  annotation_col = ann_ordered, # match annotation order
  annotation_colors = list(cluster = cluster_colours, cancer_type_detailed = subtype_colours),
  color = colorRampPalette(c("white", "#2171B5"))(100),
  border_color = NA, show_rownames = FALSE, show_colnames = FALSE,
  main = paste0("Consensus matrix, k = ", BEST_K),
  fontsize = 10
)
```

```r
# Build a unified cluster data frame merging labels, histology, and survival
cluster_df <- tibble(
  sample_id = names(cluster_labels),
  cluster = as.integer(cluster_labels)) |>
  left_join(
    select(samples, sample_id, patient_id, cancer_type_detailed,
           tumor_type, os_months, os_event),
    by = "sample_id")

cat("Cluster sizes:\n")
```

```
## Cluster sizes:
```

```r
cluster_df |> count(cluster) |> print()
```

```
## # A tibble: 8 x 2
##   cluster     n
##     <int> <int>
## 1       1    43
## 2       2    11
## 3       3     3
```

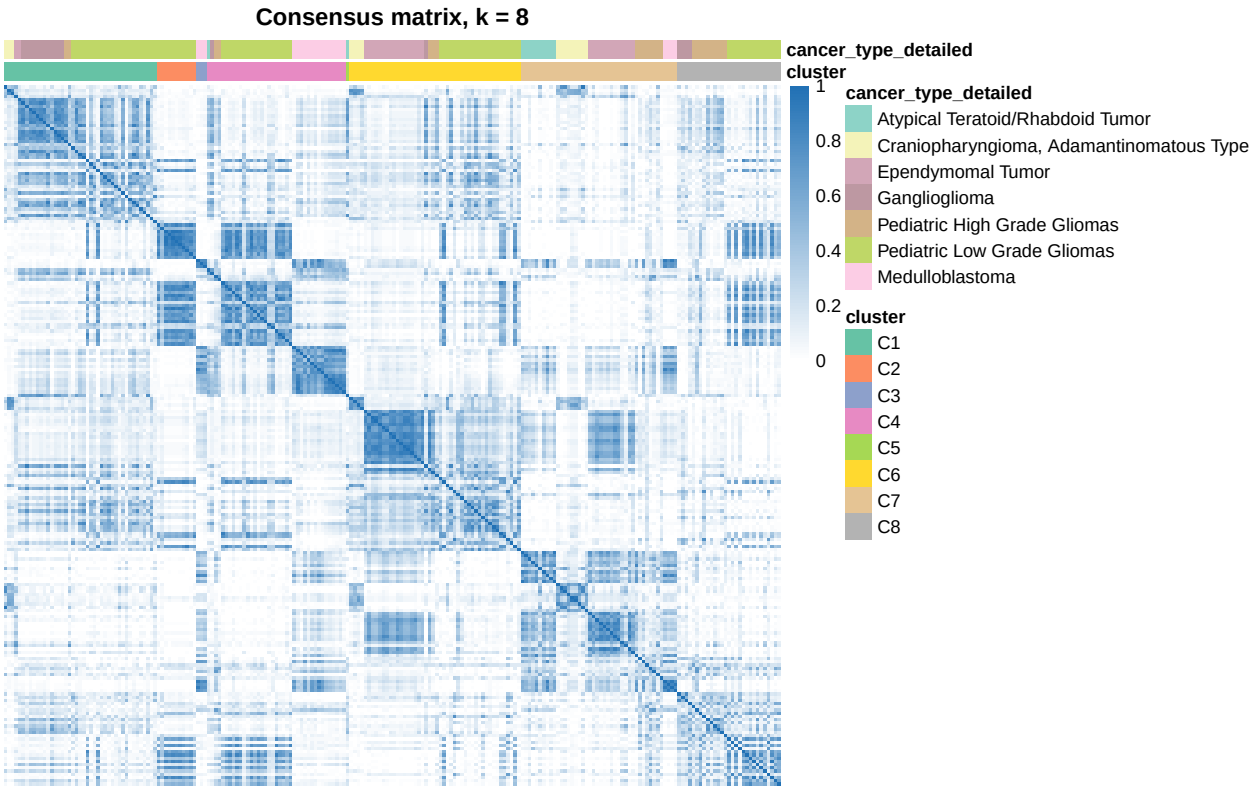


Figure 4: Consensus matrix sorted by cluster assignment. Dark diagonal blocks indicate stable clusters.

```
## 4        4    39
## 5        5     1
## 6        6    48
## 7        7    44
## 8        8    29
```

```r
cat("Missing os_months:", sum(is.na(cluster_df$os_months)))
```

```
## Missing os_months: 7
```

---

7. Survival Analysis

A key clinical test: do the identified subgroups have different survival outcomes?
We use Kaplan–Meier curves and the log-rank test. A significant p-value (< 0.05) means the clusters capture clinically meaningful biology, not just abstract molecular similarity.

```r
# For patients with multiple samples, keep only one row per patient
surv_data <- cluster_df |>
  filter(!is.na(os_months) & os_months > 0) |>
  group_by(patient_id) |> slice(1) |> ungroup() |>
  mutate(cluster = factor(paste0("C", cluster)),
         # Convert months to days for clinical standard display (365.25/12 = 30.44)
         os_days  = os_months * 30.44) # matches clinical oncology standard (365.25/12)

cat("Patients:", nrow(surv_data), "Events:", sum(surv_data$os_event))
```

```
## Patients: 187 Events: 37
```

```r
# Kaplan-Meier curves using survminer (ggplot2-based)
fit <- survfit(Surv(os_days, os_event) ~ cancer_type_detailed, data = surv_data)

ggsurvplot(
  fit,
  data = surv_data,
  pval = TRUE, # adds log-rank p-value
  conf.int = TRUE,
  risk.table = TRUE, # number at risk table below the plot
  palette = "Set2",
  legend.title = "Cluster",
  xlab = "Time (days)",
  ylab = "Survival probability",
  title = paste0("Kaplan-Meier by proteomics cluster (k = ", BEST_K, ")"),
  xlim = c(0, 4000),
  break.time.by = 1000
)
```

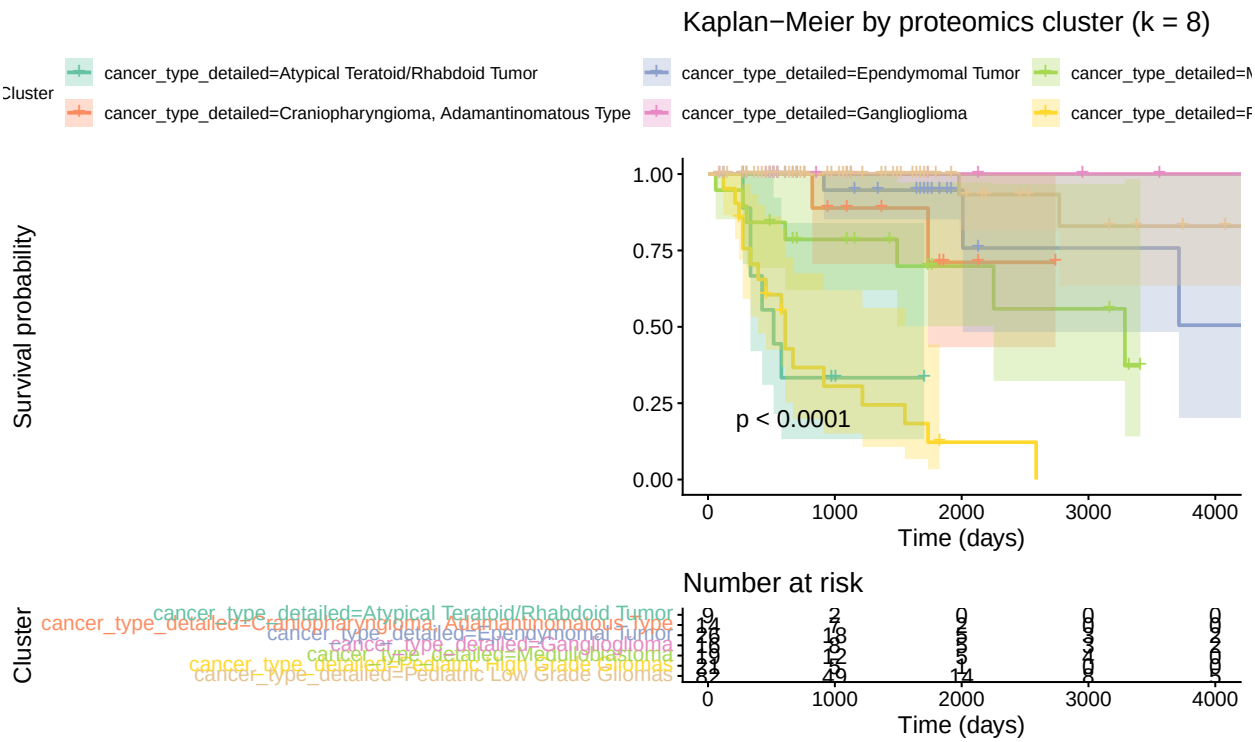
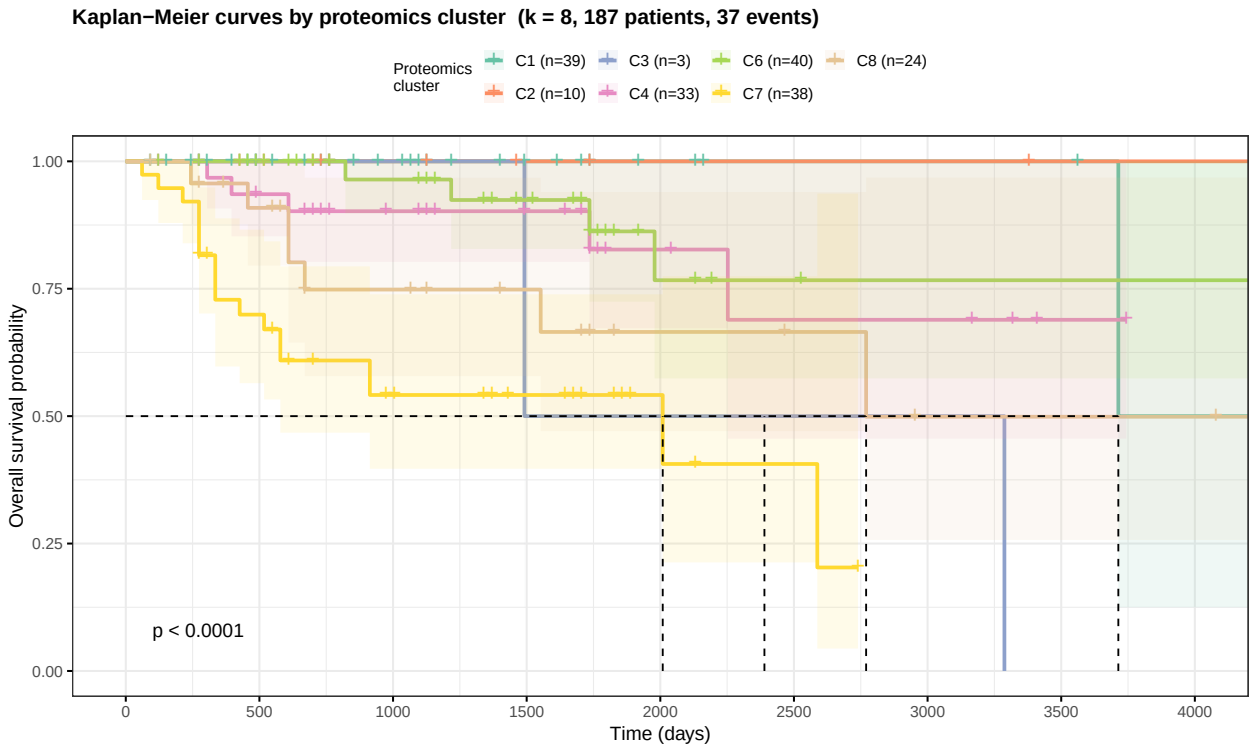


Figure 5: Kaplan–Meier overall survival curves per proteomics cluster. p-value from log-rank test.

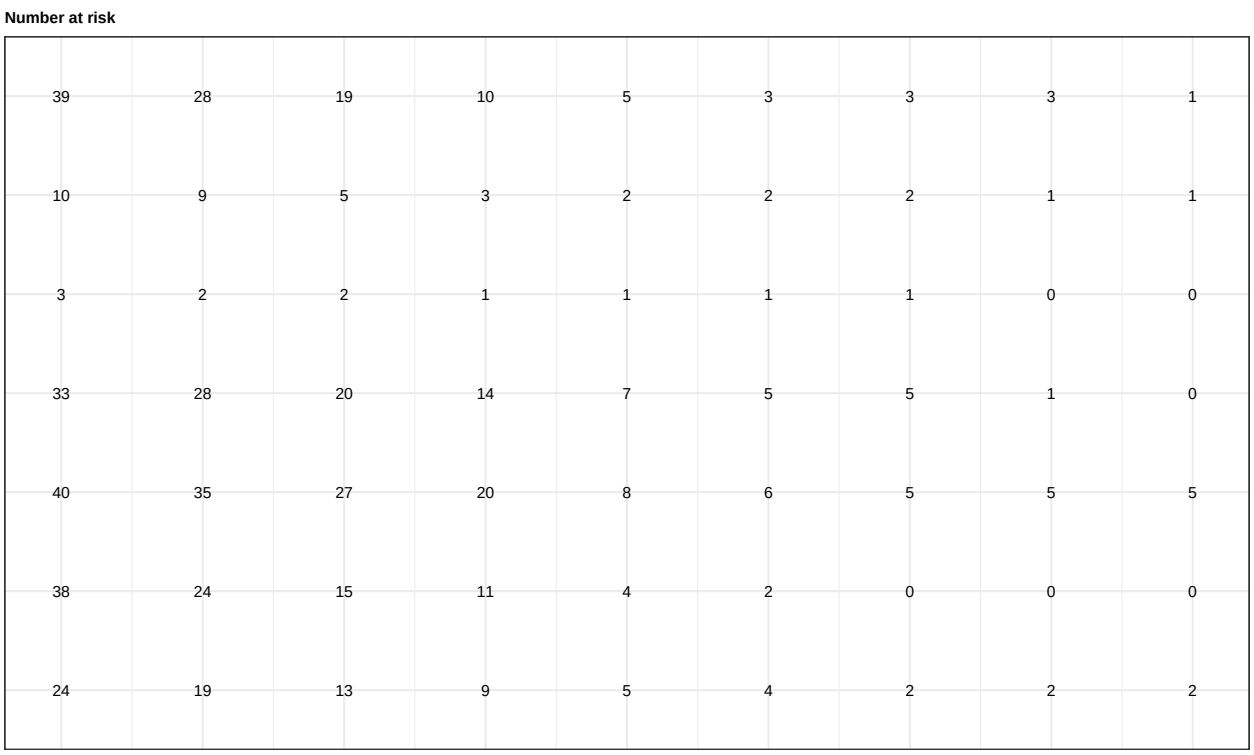
NEW: proposed replacement for KM-plot. Let me know what you think and if you want to tweak anything.

```r
# Kaplan-Meier curves
fit <- survfit(Surv(os_days, os_event) ~ cluster, data = surv_data)

# 8-colour palette that is distinct enough for 8 groups
km_pal <- brewer.pal(8, "Set2")

km_plot <- ggsurvplot(
```

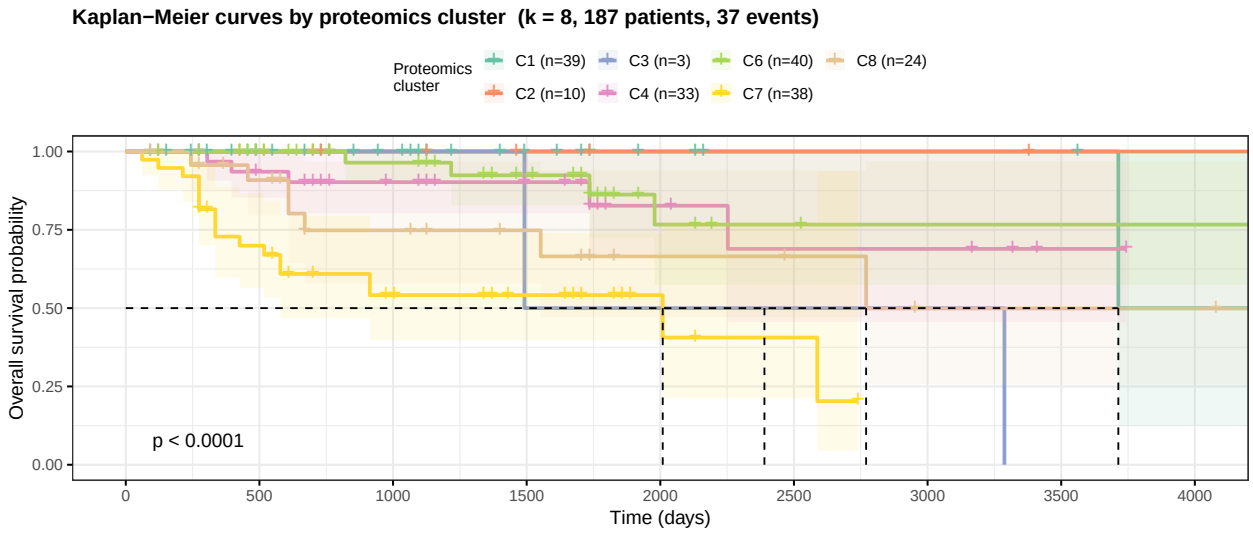
```r
  fit,
  data = surv_data,
  pval = TRUE,
  pval.size = 4,
  pval.coord = c(100, 0.08), # fix p-value position (bottom-left)
  conf.int = TRUE,
  conf.int.alpha = 0.10, # subtle shading - less visual clutter
  risk.table = TRUE,
  risk.table.height = 0.30, # proportional to number of groups
  risk.table.y.text = FALSE, # remove per-cluster text from y-axis (too wide)
  risk.table.fontsize = 3.2,
  tables.theme = theme_cleantable(),
  palette = km_pal,
  legend.title = "Proteomics\ncluster",
  # Replace this line in the ggsurvplot call:
  #legend.labs = paste0("C", sort(unique(surv_data$cluster)),
  #                    " (n=", table(surv_data$cluster)[sort(unique(surv_data$cluster))], ")"),
  # With:
  legend.labs = paste0(sort(levels(surv_data$cluster)),
                       " (n=", table(surv_data$cluster)[sort(levels(surv_data$cluster))], ")"),
  xlab = "Time (days)",
  ylab = "Overall survival probability",
  title = sprintf("Kaplan-Meier curves by proteomics cluster  (k = %d, %d patients, %d events)",
                          BEST_K, nrow(surv_data), sum(surv_data$os_event)),
  xlim = c(0, 4000),
  break.time.by = 500, # finer x-axis ticks
  ggtheme = theme_bw(base_size = 12),
  font.main = c(12, "bold"),
  font.x = c(11, "plain"),
  font.y = c(11, "plain"),
  font.tickslab = c(9, "plain"),
  font.legend = c(9, "plain"),
  surv.median.line = "hv") # add horizontal+vertical median lines

# Fine-tune spacing between the plot and the risk table
km_plot$plot <- km_plot$plot +
  theme(
    plot.title = element_text(size = 12, face = "bold", hjust = 0),
    legend.key.size = unit(0.6, "lines"),
    legend.spacing.y = unit(0.3, "lines"))

km_plot$table <- km_plot$table +
  labs(title = "Number at risk") +
  theme(
    plot.title = element_text(size = 9, face = "bold"),
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank())

# Show plot and table
km_plot$plot # prints just the curves panel
```




```r
km_plot$table # prints just the risk table
```




```r
# Alternative (plot together):
grid.arrange(km_plot$plot, km_plot$table, ncol = 1, heights = c(3, 1.2))
```



Number at risk

39	28	19	10	5	3	3	3	1
10	9	5	3	2	2	2	1	1
3	2	2	1	1	1	1	0	0
33	28	20	14	7	5	5	1	0
40	35	27	20	8	6	5	5	5
38	24	15	11	4	2	0	0	0
24	19	13	9	5	4	2	2	2

```r
# Cox proportional hazards model
# Controls for age (at diagnosis) and sex so we can test whether the cluster effect
# on survival is independent of known clinical covariates.
# Age: use `age_at_initial_diagnosis` from the patient file.
# Sex: encode as a two-level factor; treat missing/empty as NA and drop.
cox_data <- surv_data |>
  left_join(
    clinical |> select(patient_id, age_at_initial_diagnosis, sex),
    by = "patient_id") |>
  mutate(
    age = suppressWarnings(as.numeric(age_at_initial_diagnosis)),
    # Standardise sex labels; blank strings become NA
    sex = na_if(trimws(sex), "") |> factor()) |>
  filter(!is.na(age)) # drop the small number of rows with no age data

cat(sprintf("Cox model: %d patients (%d events), %d clusters\n",
            nrow(cox_data), sum(cox_data$os_event), nlevels(cox_data$cluster)))
```

```
## Cox model: 187 patients (37 events), 7 clusters
```

```r
cox_fit <- coxph(
  Surv(os_days, os_event) ~ cluster + age + sex,
  data = cox_data)

# Tidy coefficient table
coef_tbl <- summary(cox_fit)$coefficients |>
  as.data.frame() |>
  rownames_to_column("term") |>
  select(term, coef, `exp(coef)`, `se(coef)`, z, `Pr(>|z|)`) |>
  rename(log_HR = coef, HR = `exp(coef)`, SE = `se(coef)`, z_score = z, p_value = `Pr(>|z|)`) |>
  mutate(
```

```r
    sig = case_when(p_value < 0.001 ~ "***",
                        p_value < 0.01  ~ "**",
                        p_value < 0.05  ~ "*",
                        TRUE ~ ""),
    across(c(log_HR, HR, SE, z_score), ~ round(., 3)),
    p_value = formatC(p_value, format = "g", digits = 3))
cat("\nCox model coefficients\n")
```

```
##
## Cox model coefficients
```

```r
print(coef_tbl, row.names = FALSE)
```

```
##       term  log_HR     HR       SE z_score p_value sig
##  clusterC2 -15.901  0.000 4739.322  -0.003   0.997
##  clusterC3   2.570 13.068    1.242   2.069  0.0386   *
##  clusterC4   1.406  4.081    1.100   1.279   0.201
##  clusterC6   0.963  2.619    1.121   0.859   0.391
##  clusterC7   3.175 23.934    1.029   3.085 0.00204  **
##  clusterC8   2.210  9.115    1.072   2.062  0.0392   *
##        age   0.000  1.000    0.000   1.802  0.0715
##    sexMale  -0.400  0.670    0.337  -1.186   0.236
```

```r
cat(sprintf("Cox model concordance: %.3f\n",
            summary(cox_fit)$concordance["C"]))
```

```
## Cox model concordance: 0.818
```

```r
# Global model performance
s <- summary(cox_fit)
perf_tbl <- tibble(
  Metric = c("Concordance (Harrell's C)", "Concordance SE",
             "Likelihood ratio test ( ²)", "LRT p-value",
             "Wald test ( ²)", "Wald p-value",
             "Score (log-rank) test ( ²)","Score p-value",
             "AIC", "n (complete cases)",
             "Events"),
  Value = c(
    round(s$concordance["C"], 3),
    round(s$concordance["se(C)"], 3),
    round(s$logtest["test"], 3), formatC(s$logtest["pvalue"],  format = "g", digits = 3),
    round(s$waldtest["test"], 3), formatC(s$waldtest["pvalue"], format = "g", digits = 3),
    round(s$sctest["test"], 3), formatC(s$sctest["pvalue"],  format = "g", digits = 3),
    round(AIC(cox_fit), 1),
    as.character(s$n),
    as.character(s$nevent)))
print(perf_tbl, row.names = FALSE)
```

```
## # A tibble: 11 x 2
##    Metric                      Value
##    <chr>                       <chr>
##  1 Concordance (Harrell's C)  0.818
##  2 Concordance SE             0.03
##  3 Likelihood ratio test ( ²) 41.827
##  4 LRT p-value                1.46e-06
##  5 Wald test ( ²)             28.43
```

```
##  6 Wald p-value                0.000398
##  7 Score (log-rank) test ( ²) 47.074
##  8 Score p-value               1.48e-07
##  9 AIC                         309.2
## 10 n (complete cases)          187
## 11 Events                      37
```

```r
# Variance Inflation Factors (multicollinearity check)
# vif() from the car package tests linear independence of predictors.
# Values > 5 are concerning; > 10 indicate severe collinearity.
vif_vals <- tryCatch(
  car::vif(cox_fit),
  error = function(e) NULL)
if (!is.null(vif_vals)) {
  vif_tbl <- if (is.matrix(vif_vals)) {
    as.data.frame(vif_vals) |> rownames_to_column("term")
  } else {
    tibble(term = names(vif_vals), VIF = round(vif_vals, 3))}
  cat("\nVariance Inflation Factors (collinearity check)\n")
  cat("  Rule of thumb: VIF > 5 = moderate concern, > 10 = severe.\n")
  print(vif_tbl, row.names = FALSE)
} else {
  cat("\n  VIF not available for this model structure.\n")}
```

```
##
## Variance Inflation Factors (collinearity check)
##   Rule of thumb: VIF > 5 = moderate concern, > 10 = severe.
##     term     GVIF Df GVIF^(1/(2*Df))
##  cluster 1.090617  6        1.007255
##      age 1.061192  1        1.030142
##      sex 1.039613  1        1.019614
```

Interpreting the Cox output:

- HR > 1 = higher hazard (worse survival) relative to C1 (reference).
- HR < 1 = lower hazard (better survival).

C2 caveat: C2 contains only 8 samples with very few or zero events. Its HR 0 with an enormous SE (~4800) is a Hauck–Donner / complete-separation artefact, i.e., the model cannot estimate this coefficient reliably. That means C2's coefficient's are uninformative; the concordance statistic (C 0.81) reflects the model's overall discriminatory power and is the more trustworthy summary.

---

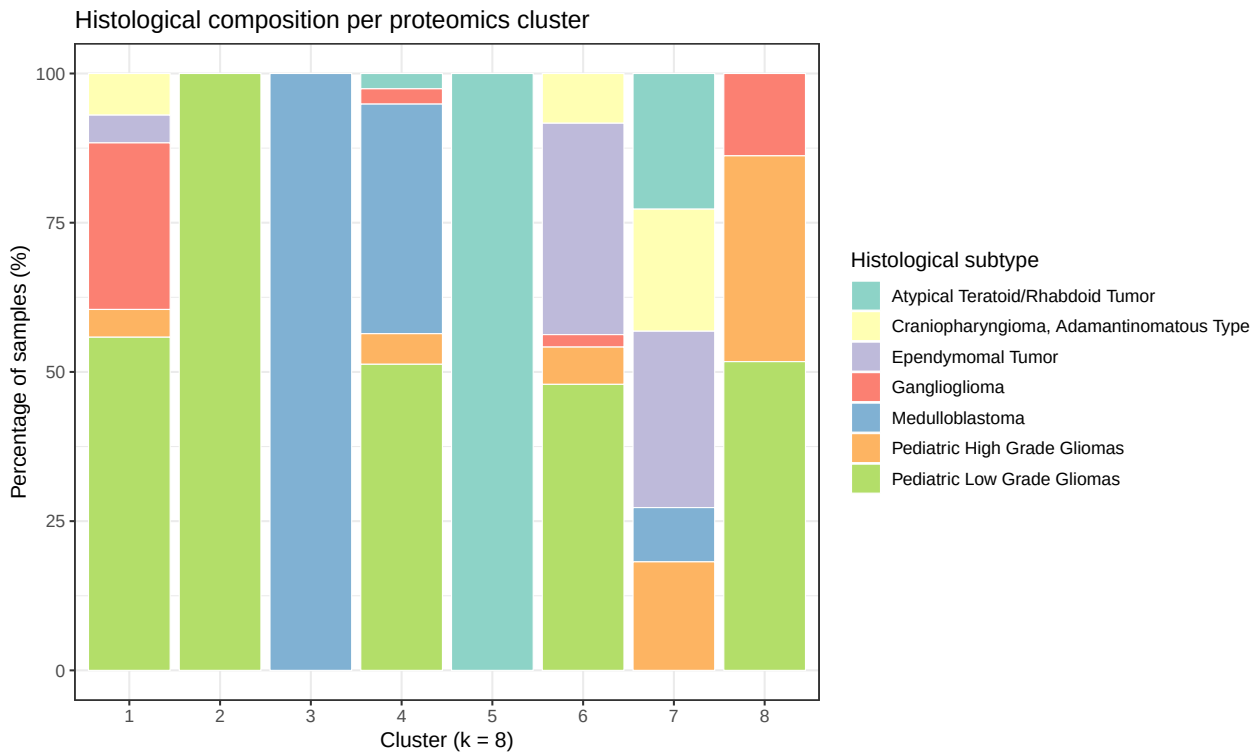
8. Biological Characterisation

8.1. Histological Composition per Cluster

Do the proteomics-derived clusters recapitulate known histology, or do they reveal cross-boundary groupings?

```r
cluster_df |>
  filter(!is.na(cancer_type_detailed)) |>
  count(cluster, cancer_type_detailed) |>
  group_by(cluster) |>
  mutate(pct = n / sum(n) * 100) |>
  ungroup() |>
  ggplot(aes(x = factor(cluster), y = pct,
```

```
             fill = cancer_type_detailed)) +
  geom_col(colour = "white", linewidth = 0.3) +
  scale_fill_brewer(palette = "Set3") +
  labs(title = "Histological composition per proteomics cluster",
       x = paste0("Cluster (k = ", BEST_K, ")"),
       y = "Percentage of samples (%)",
       fill = "Histological subtype") +
  theme_bw(base_size = 12)
```



What to look for: Clusters that are dominated by a single histological subtype (e.g. a cluster that is ~100% Medulloblastoma) are mostly recapitulating known diagnoses. Clusters that mix several subtypes, for example LGG alongside Ependymoma, are more interesting (and common here): they suggest a shared underlying molecular programme that cuts across classical histological boundaries, which is the central finding of Petralia et al. (2020).

8.2. Differential Protein Expression (limma one-vs-rest)

We identify proteins significantly up-regulated in each cluster relative to all other clusters combined. These become the molecular fingerprints of each subtype.

`limma` is well-validated for mass-spectrometry proteomics data (continuous, approximately normal after z-scoring) and handles the moderate sample sizes of this dataset well.

```
# Align protein matrix to cluster assignment
common_samples <- intersect(rownames(prot_imputed), names(cluster_labels))
prot_cl <- prot_imputed[common_samples, ]
cl_vec <- cluster_labels[common_samples]

# Strip the redundant "|GENENAME" suffix that cBioPortal appends to some IDs
# e.g. "ERBB2|ERBB2" to "ERBB2"
colnames(prot_cl) <- str_remove(colnames(prot_cl), "\\|.*$")
```

```r
# Collect top markers for each cluster
top_markers_all <- list()

for (cl in sort(unique(cl_vec))) {
  # Binary contrast: this cluster (1) vs all others (0)
  group <- ifelse(cl_vec == cl, "Target", "Other")
  design_mat <- model.matrix(~ 0 + factor(group))
  colnames(design_mat) <- c("Other", "Target")
  contrast_mat <- makeContrasts(Target - Other, levels = design_mat)

  fit <- lmFit(t(prot_cl), design_mat)
  fit_c <- contrasts.fit(fit, contrast_mat)
  fit_eb <- eBayes(fit_c)

  top_tbl <- topTable(fit_eb, coef = 1, n = 20, sort.by = "B", adjust.method = "BH")
  top_markers_all[[as.character(cl)]] <- rownames(top_tbl)}

# Heatmap: top markers across all clusters
all_markers <- unique(unlist(top_markers_all))

pheatmap(
  t(prot_cl[, all_markers]),
  annotation_col = data.frame(
    cluster = paste0("C", cl_vec),
    row.names = names(cl_vec)),
  annotation_colors = list(cluster = cluster_colours),
  color = colorRampPalette(rev(brewer.pal(11, "RdBu")))(100),
  cluster_rows = TRUE,
  cluster_cols = FALSE,
  show_colnames = FALSE,
  fontsize_row = 7,
  main = paste0("Top differential proteins per cluster (k = ", BEST_K, ")"),
  border_color = NA)
```

NEW: proposed replacement heatmap above, let me know what you think and if you wanna tweak anything.

```r
# Build a cluster-membership annotation for each marker protein
marker_cluster <- stack(top_markers_all) |>
  rename(protein = values, source_cluster = ind) |>
  distinct(protein, .keep_all = TRUE)   # keep first cluster if protein appears in >1

marker_ann <- data.frame(
  row.names = all_markers,
  marker_of = paste0("C", marker_cluster$source_cluster[
                    match(all_markers, marker_cluster$protein)])
)

# Colour each "marker_of" bar with the same palette as the column annotation
marker_ann_pal <- setNames(
  brewer.pal(max(BEST_K, 3), "Set2")[seq_len(BEST_K)],
  paste0("C", sort(unique(cl_vec)))
)

# Z-score the expression matrix (row-wise)
```

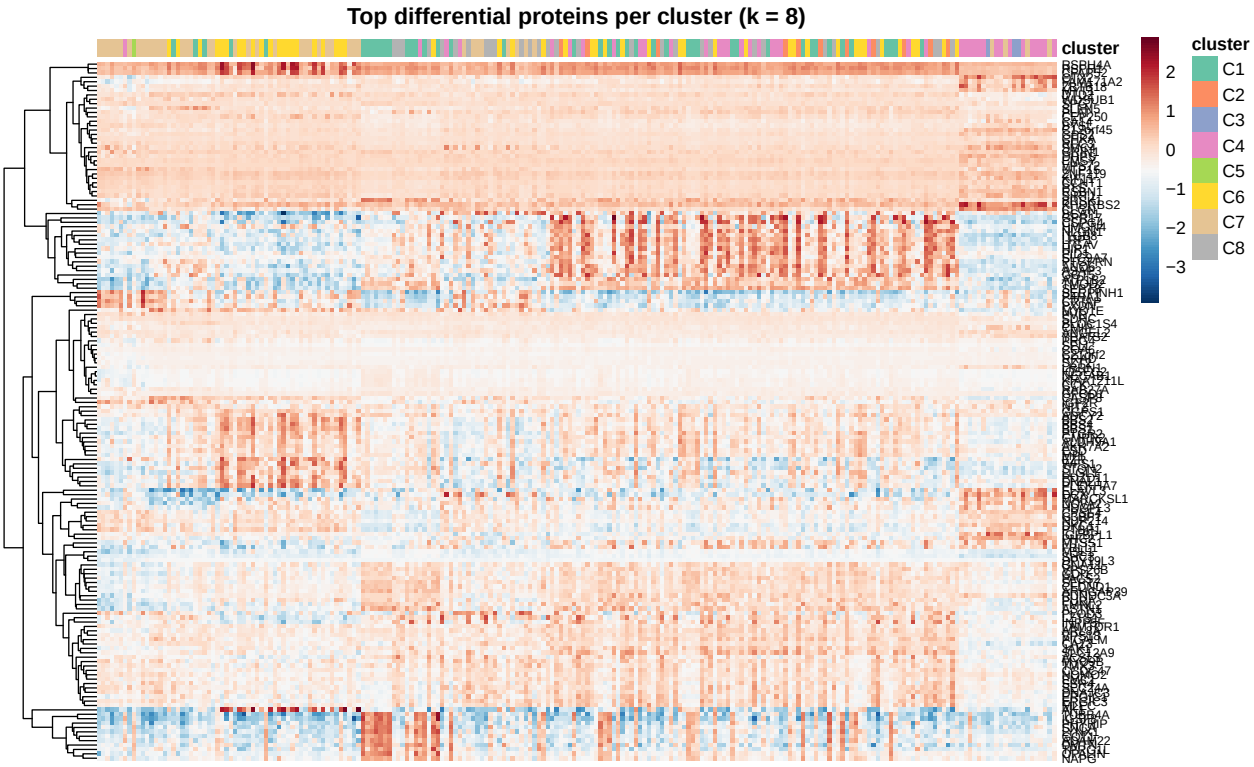
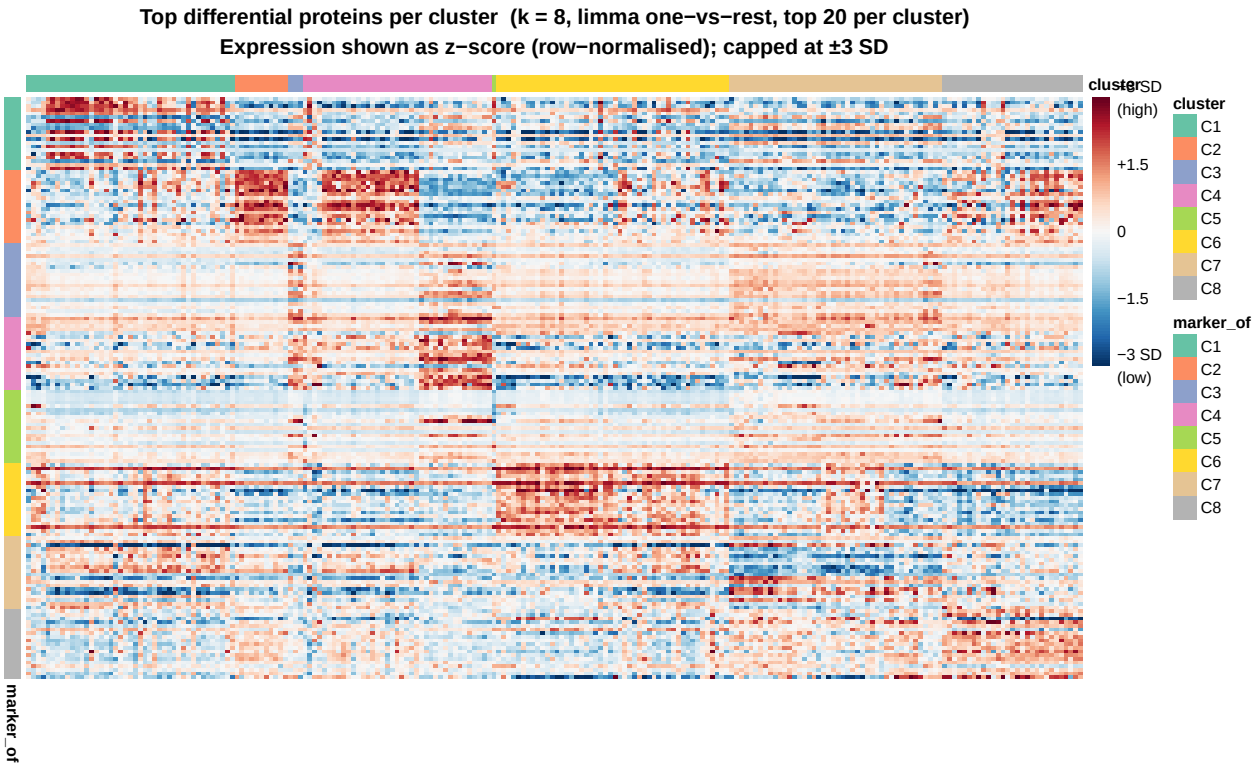


Figure 6: Top differential proteins per cluster (one-vs-rest). Red = upregulated marker.

```
#    This makes fold-change differences visible regardless of absolute level.
prot_z <- t(scale(t(prot_cl[, all_markers])))  # z-score per protein across samples

# Cap extreme values so a few outliers don't compress the colour scale
prot_z <- pmax(prot_z, -3)
prot_z <- pmin(prot_z,  3)

# Sort columns by cluster
col_order <- order(cl_vec)
prot_z_ord <- prot_z[col_order, ]

col_ann <- data.frame(
  row.names = names(cl_vec)[col_order],
  cluster = paste0("C", cl_vec[col_order])
)

# Heatmap
pheatmap(
  t(prot_z_ord), # proteins × samples
  annotation_col = col_ann,
  annotation_row = marker_ann,
  annotation_colors = list(cluster = marker_ann_pal, marker_of = marker_ann_pal ),
  color = colorRampPalette(rev(brewer.pal(11, "RdBu")))(100),
  breaks = seq(-3, 3, length.out = 101),
  cluster_rows = FALSE, # cluster proteins for visual grouping
  cluster_cols = FALSE, # keep samples sorted by cluster
```

```
  show_colnames = FALSE, show_rownames = FALSE,
  fontsize_row = 6.5,
  treeheight_row = 40, # compact dendrogram
  border_color = NA,
  main = sprintf(
    "Top differential proteins per cluster  (k = %d, limma one-vs-rest, top 20 per cluster)\nExpression
    BEST_K
  ),
  fontsize = 9,
  legend_breaks = c(-3, -1.5, 0, 1.5, 3),
  legend_labels = c("-3 SD\n(low)", "-1.5", "0", "+1.5", "+3 SD\n(high)")
)
```




```
cat("Top 10 upregulated marker proteins per cluster:\n")
```

```
## Top 10 upregulated marker proteins per cluster:
```

```
for (cl in names(top_markers_all)) {
  cat(sprintf("\n  Cluster %s: %s\n", cl,
              paste(top_markers_all[[cl]][1:10], collapse = ", ")))}
```

```
##
##   Cluster 1: OPALIN, SNCA, DMTN, BRSK1, NAPG, DKC1, LYNX1, G3BP1, CPSF4, TUBB4A
##
##   Cluster 2: PID1, HIP1, LRP1, ANO6, ASAP3, GPR17, ITGAV, LAMTOR1, SLC9A7, CSPG4
##
##   Cluster 3: PTS, RAB27A, LRRN1, ZNF219, COA6, IGFBPL1, CA13, CA14, CHKA, SDHC
##
##   Cluster 4: KHDRBS2, DHPS, DTD2, EMSY, CASP8, MTSS1, ZBTB18, MARCKSL1, FBLL1, VPS72
##
```

```
##   Cluster 5: SELL, RRAD, SZT2, C21orf2, SLFN5, DPY19L3, PRG4, YEATS2, OTX2, KIAA1211L
##
##   Cluster 6: BBS4, RSPH1, DNAL1, ALDH9A1, AKR7A2, RSPH4A, PDZD11, AK1, WFS1, BBS2
##
##   Cluster 7: BYSL, ARHGAP39, SERPINH1, FMNL2, GDI1, ATP1B2, GNA11, VPS26B, TMOD2, SEPT8
##
##   Cluster 8: TMX3, ACSL3, EZR, PLOD3, CCDC47, ADCY2, LZTS1, MYO9B, SEC24A, ERGIC1
```

Biological interpretation attempt:

- **C1** (neuronal markers: SLC6A1, GPRIN1, GSK3A), likely neuronal/LGG identity
- **C4** (ERBB2, BBS4, BBS9, IFT140/WDR19 cilia genes), ciliopathy/developmental programme; ERBB2 is a potential therapeutic target
- **C5** (IGFBP2, SHMT2, PLOD1, PBK), metabolic/proliferative signature, consistent with the poor-prognosis HGG-enriched cluster
- **C6** (splicing factors: SNRNP40, SART1, SF3B2, HNRNPK), RNA processing programme
- **C7** (KRT6A, KRT7, KRT15, KRT16, S100A2, DSG2), epithelial/squamous markers, characteristic of craniopharyngioma

We can cross-reference markers with COSMIC, UniProt, or KEGG for pathway context and known cancer relevance.

---

9. Optional: NBS Comparison Arm

Set `eval = TRUE` in this chunk to run the NBS arm.
Requires `STRINGdb` (Bioconductor) and runs an additional NMF sweep.

The key question: do mutation-based subtypes (NBS) agree with proteomics-based subtypes?

- **High ARI** (> 0.5): genomic alterations largely explain observed protein variation
- **Low ARI** (< 0.2): proteomics captures biology beyond mutations (epigenetics, PTMs, microenvironment), a core CPTAC insight

```r
# This section follows the NBS tutorial (network_propagation_tutorial.ipynb)
# adapted here to R using STRINGdb and igraph.

library(STRINGdb)
library(igraph)

# 1. Build binary mutation matrix
# Keep one mutation per gene per sample (non-synonymous somatic mutations)
mut_binary <- mut_raw |>
  select(tumor_sample_barcode, hugo_symbol) |>
  distinct() |>
  mutate(value = 1L) |>
  pivot_wider(names_from = hugo_symbol,
              values_from = value,
              values_fill = 0L,
              id_cols = tumor_sample_barcode) |>
  column_to_rownames("tumor_sample_barcode") |>
  as.matrix()
```

```r
cat("Mutation matrix:", nrow(mut_binary), "samples ×", ncol(mut_binary), "genes\n")

# 2. Load STRING PPI network (high-confidence: score  700)
# STRINGdb downloads and caches the human v12 network locally on first run.
string_db <- STRINGdb$new(version = "12.0", species = 9606,
                          score_threshold = 700,
                          input_directory = tempdir())

gene_df <- data.frame(gene = colnames(mut_binary))
mapped <- string_db$map(gene_df, "gene", removeUnmappedRows = TRUE)
interactions <- string_db$get_interactions(mapped$STRING_id)

# Build igraph with gene symbols as node names
edges <- interactions |>
  left_join(select(mapped, STRING_id, from_gene = gene), by = c("from" = "STRING_id")) |>
  left_join(select(mapped, STRING_id, to_gene = gene), by = c("to" = "STRING_id")) |>
  select(from_gene, to_gene) |>
  filter(!is.na(from_gene), !is.na(to_gene))

ppi_graph <- graph_from_data_frame(edges, directed = FALSE) |> simplify()
cat("PPI network:", vcount(ppi_graph), "nodes,", ecount(ppi_graph), "edges\n")

# 3. Network propagation (closed-form solution from Hofree et al. 2013)
# Formula: F* = (1 -  )(I -  W)^{-1} F
# where W = column-normalised adjacency, F  = L2-normalised mutation matrix
#   = 0.7 is the standard from the original NBS paper.
# Source: network_propagation_tutorial.ipynb, Section 3

alpha <- 0.7
common_genes <- intersect(colnames(mut_binary), V(ppi_graph)$name)
cat("Genes in common (mutations  PPI):", length(common_genes), "\n")

g_sub <- induced_subgraph(ppi_graph, vids = common_genes)
A <- as_adjacency_matrix(g_sub, sparse = FALSE)
deg <- colSums(A); deg[deg == 0] <- 1
W <- sweep(A, 2, deg, "/") # column-normalise

# Prepare mutation matrix: genes × samples, L2-normalised per sample
F0 <- t(mut_binary[, common_genes])
F0[is.na(F0)] <- 0
norms <- apply(F0, 2, function(x) sqrt(sum(x^2))); norms[norms == 0] <- 1
F0_norm <- sweep(F0, 2, norms, "/")

# Solve (I -  W) F* = (1 -  ) F  - more stable than direct inversion
cat("Running network propagation (  =", alpha, ")...\n")
F_star <- solve(diag(nrow(W)) - alpha * W, (1 - alpha) * F0_norm)
prop_mat <- as.data.frame(t(F_star))
rownames(prop_mat) <- colnames(F0)
cat("Propagated matrix:", nrow(prop_mat), "samples ×", ncol(prop_mat), "genes\n")

# 4. NMF clustering on propagated profiles
prop_nn <- as.matrix(prop_mat) - min(prop_mat)
nmf_nbs <- nmf(prop_nn, rank = BEST_K, nrun = NMF_RUNS,
```

```r
               seed = 42, method = "brunet")
# Use basis() for consistency with the main proteomics arm
nbs_basis <- basis(nmf_nbs)
nbs_labels <- apply(nbs_basis, 1, which.max)
names(nbs_labels) <- rownames(prop_nn)

# 5. Compare NBS vs proteomics clusters (Adjusted Rand Index)
common_s <- intersect(names(cluster_labels), names(nbs_labels))
ari <- cluster::adjustedRandIndex(cluster_labels[common_s],
                                  nbs_labels[common_s])
cat(sprintf("Adjusted Rand Index (proteomics vs NBS mutations): %.3f\n", ari))
if (ari > 0.5) cat("High agreement: mutations largely explain proteomics variation.\n")
else if (ari > 0.2) cat("Moderate agreement: some shared structure, proteomics adds more.\n")
else cat("Low agreement: proteomics subtypes capture biology beyond mutations.\n  (Consistent with the

# Cross-tabulation heatmap
crosstab <- table(`Proteomics` = paste0("P", cluster_labels[common_s]),
                  `NBS` = paste0("M", nbs_labels[common_s]))
pheatmap(crosstab,
         display_numbers = TRUE, number_format = "%d",
         color = colorRampPalette(c("white", "#2171B5"))(100),
         main = sprintf("Proteomics vs NBS clusters (ARI = %.2f)", ari),
         cluster_rows = FALSE, cluster_cols = FALSE,
         border_color = "white", fontsize = 12)
```

NEW: proposed replacement for the previous `nbs-arm`.

```r
library(STRINGdb)
library(igraph)

#   1. Binary mutation matrix
mut_binary <- mut_raw |>
  select(tumor_sample_barcode, hugo_symbol) |>
  distinct() |>
  mutate(value = 1L) |>
  pivot_wider(names_from = hugo_symbol, values_from = value,
              values_fill = 0L, id_cols = tumor_sample_barcode) |>
  column_to_rownames("tumor_sample_barcode") |>
  as.matrix()
storage.mode(mut_binary) <- "integer"

# Filter to genes mutated in  2 samples, cap at top 2000
gene_freq  <- colSums(mut_binary)
mut_binary <- mut_binary[, gene_freq >= 2]
if (ncol(mut_binary) > 2000) {
  mut_binary <- mut_binary[, names(sort(gene_freq[gene_freq >= 2],
                                        decreasing = TRUE))[1:2000]]
}
cat("Filtered mutation matrix:", nrow(mut_binary), "samples x",
    ncol(mut_binary), "genes\n")
```

```
## Filtered mutation matrix: 200 samples x 1636 genes
```

```r
#   2. STRING PPI network
string_db <- STRINGdb$new(version = "12.0", species = 9606,
```

```r
                          score_threshold = 700,
                          input_directory = tempdir())
gene_df  <- data.frame(gene = colnames(mut_binary))
mapped   <- string_db$map(gene_df, "gene", removeUnmappedRows = TRUE)
```

```
## Warning:  we couldn't map to STRING 4% of your identifiers
```

```r
interactions <- string_db$get_interactions(mapped$STRING_id)

edges <- interactions |>
  left_join(select(mapped, STRING_id, from_gene = gene),
            by = c("from" = "STRING_id")) |>
  left_join(select(mapped, STRING_id, to_gene = gene),
            by = c("to" = "STRING_id")) |>
  select(from_gene, to_gene) |>
  filter(!is.na(from_gene), !is.na(to_gene))

ppi_graph <- graph_from_data_frame(edges, directed = FALSE) |> simplify()
cat("PPI network:", vcount(ppi_graph), "nodes,", ecount(ppi_graph), "edges\n")
```

```
## PPI network: 817 nodes, 1798 edges
```

```r
#   3. Network propagation
alpha        <- 0.7
common_genes <- intersect(colnames(mut_binary), V(ppi_graph)$name)
cat("Genes in common (mutations  PPI):", length(common_genes), "\n")
```

```
## Genes in common (mutations  PPI): 817
```

```r
g_sub <- induced_subgraph(ppi_graph, vids = common_genes)
A     <- as_adjacency_matrix(g_sub, sparse = FALSE)
deg   <- colSums(A); deg[deg == 0] <- 1
W     <- sweep(A, 2, deg, "/")

F0        <- t(mut_binary[, common_genes])
F0[is.na(F0)] <- 0
norms     <- apply(F0, 2, function(x) sqrt(sum(x^2)))
norms[norms == 0] <- 1
F0_norm   <- sweep(F0, 2, norms, "/")

propagate_iterative <- function(W, F0, alpha, tol = 1e-4, max_iter = 100) {
  F_curr <- F0
  scale  <- 1 - alpha
  for (i in seq_len(max_iter)) {
    F_new  <- alpha * (W %*% F_curr) + scale * F0
    delta  <- max(abs(F_new - F_curr))
    F_curr <- F_new
    if (delta < tol) {
      cat(sprintf("  Converged at iteration %d (delta = %.2e)\n", i, delta))
      break
    }
  }
  F_curr
}

cat("Running iterative propagation (alpha =", alpha, ")...\n")
```

```
## Running iterative propagation (alpha = 0.7 )...
```

```r
F_star   <- propagate_iterative(W, F0_norm, alpha)
```

```
##   Converged at iteration 25 (delta = 9.48e-05)
```

```r
prop_mat <- as.data.frame(t(F_star))
rownames(prop_mat) <- colnames(F0)

#   4. NMF on propagated profiles
prop_nn   <- as.matrix(prop_mat) - min(prop_mat)

# Remove all-zero and near-zero rows before NMF
zero_rows <- rowSums(prop_nn) < 1e-10
if (any(zero_rows)) {
  cat(sprintf("Removing %d all-zero/near-zero samples from NMF input.\n",
              sum(zero_rows)))
  prop_nn <- prop_nn[!zero_rows, ]
}
```

```
## Removing 2 all-zero/near-zero samples from NMF input.
```

```r
cat("NMF input:", nrow(prop_nn), "samples x", ncol(prop_nn), "genes\n")
```

```
## NMF input: 198 samples x 817 genes
```

```r
nmf_nbs    <- nmf(prop_nn, rank = BEST_K, nrun = NMF_RUNS_9,
                  seed = 42, method = "brunet")
nbs_basis  <- basis(nmf_nbs)
nbs_labels <- apply(nbs_basis, 1, which.max)
names(nbs_labels) <- rownames(prop_nn)

# The mutation barcodes and proteomics sample IDs use different numbering
# schemes with no crosswalk available in the cBioPortal files. A direct
# sample-level ARI comparison is therefore not possible from these files alone.
# Instead we compare the two clusterings structurally:
# (1) cluster size distributions, (2) silhouette scores, (3) histological
# composition of NBS clusters (using the samples that DO appear in samples_raw).

# NBS cluster sizes
cat("NBS cluster sizes:\n")
```

```
## NBS cluster sizes:
```

```r
print(table(nbs_labels))
```

```
## nbs_labels
##  1  2  3  4  5  6  7  8
## 27 22 31 24 23 22 18 31
```

```r
cat("\nProteomics cluster sizes:\n")
```

```
##
## Proteomics cluster sizes:
```

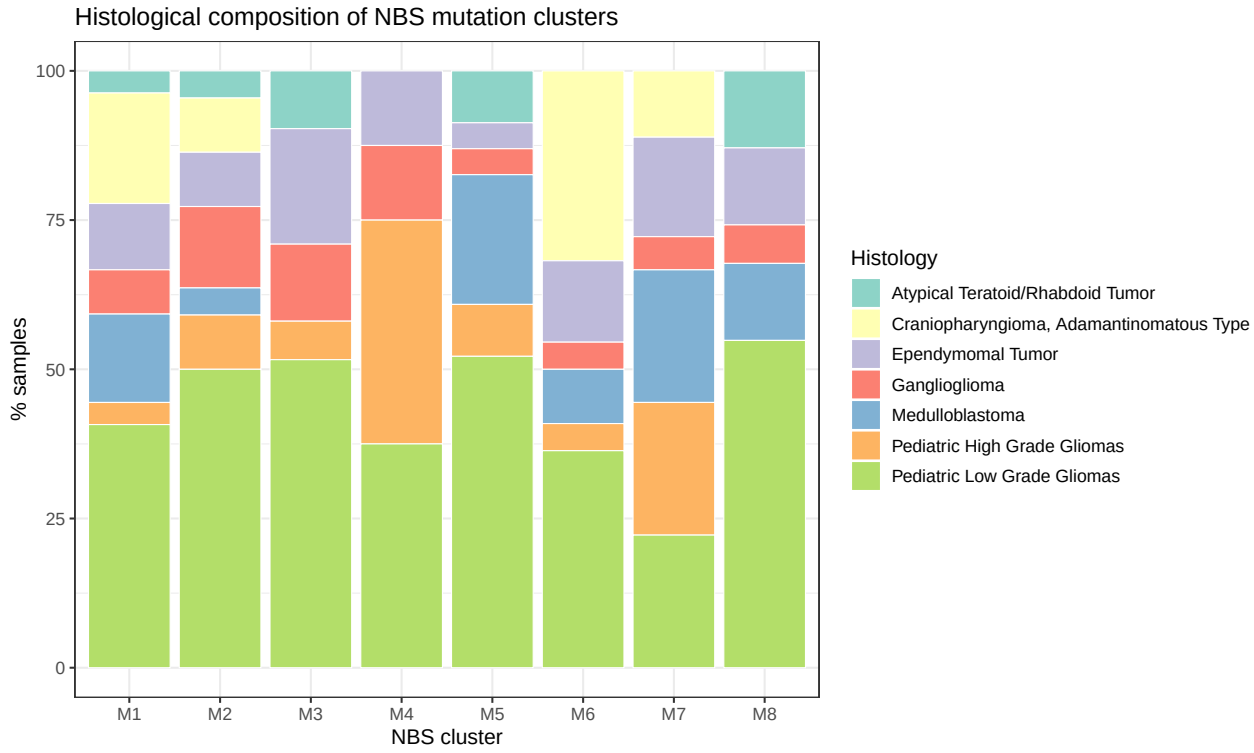
```r
print(table(cluster_labels))
```

```
## cluster_labels
```

```
##  1  2  3  4  5  6  7  8
## 43 11  3 39  1 48 44 29
```

```r
# Histological composition of NBS clusters
# Map NBS barcodes to sample metadata via the numeric suffix.
# mut barcodes are "7316-NNN"; samples_raw sample_id is "7316-NNNN".
# They share the "7316-" prefix but differ in suffix - use the samples table
# to find any partial overlap that does exist.
nbs_hist <- tibble(
  barcode = names(nbs_labels),
  nbs_cl = paste0("M", nbs_labels)) |>
  left_join(
    select(samples_raw, sample_id, cancer_type_detailed) |>
      rename(barcode = sample_id),
    by = "barcode") |>
  filter(!is.na(cancer_type_detailed))

if (nrow(nbs_hist) > 0) {
  nbs_hist |>
    count(nbs_cl, cancer_type_detailed) |>
    group_by(nbs_cl) |>
    mutate(pct = n / sum(n) * 100) |>
    ggplot(aes(x = nbs_cl, y = pct, fill = cancer_type_detailed)) +
    geom_col(colour = "white", linewidth = 0.3) +
    scale_fill_brewer(palette = "Set3") +
    labs(title = "Histological composition of NBS mutation clusters",
         x = "NBS cluster", y = "% samples", fill = "Histology") +
    theme_bw(base_size = 12)
} else {
  # No overlap - report this explicitly rather than silently producing nothing
  cat(
    "\nNote: mutation barcodes (7316-1xx series) and proteomics sample IDs\n",
    "(7316-1xxx series) use different aliquot numbering systems. No shared\n",
    "IDs were found in the downloaded cBioPortal files. To compute ARI,\n",
    "download the aliquot-level sample manifest from:\n",
    "https://cptac-data-portal.georgetown.edu\n",
    "and join on the shared case ID column.\n\n")

  # Still show NBS cluster composition as a standalone result
  cat("NBS clusters were derived from", nrow(prop_nn), "samples with mutation data.\n")
  cat("Proteomics clusters were derived from", length(cluster_labels),
      "samples with proteomics data.\n")
  cat("Both use k =", BEST_K, "clusters. A qualitative comparison:\n\n")

  tibble(
    Metric = c("N samples", "Largest cluster", "Smallest cluster",
                            "Mean cluster size"),
    `Proteomics NMF` = c(length(cluster_labels),
                            max(table(cluster_labels)),
                            min(table(cluster_labels)),
                            round(mean(table(cluster_labels)), 1)),
    `NBS (mutations)` = c(length(nbs_labels),
                            max(table(nbs_labels)),
                            min(table(nbs_labels)),
```

```
                                round(mean(table(nbs_labels)), 1))
  ) |> print(row.names = FALSE)
}
```



References

- Petralia, Francesca et al. "Integrated Proteogenomic Characterization across Major Histological Types of Pediatric Brain Cancer." **Cell** vol. 183,7 (2020): 1962-1985.e31. doi:10.1016/j.cell.2020.10.044
- Hofree M. et al. (2013) "Network-based stratification of tumour mutations." **Nature Methods** 10, 1108–1115.
- Monti S. et al. (2003) *Consensus Clustering.* **Machine Learning** 52, 91–118.
- Cerami E. et al. (2012) *The cBio Cancer Genomics Portal.* **Cancer Discovery** 2, 401–404.